

Organizing Our Content

You know how in the last part we mentioned “*in-line*” styling. That was ACTUALLY us using CSS properties, but inside the HTML line; thus why it’s called in-line.

CSS doesn’t only do editing for the text, we can also edit A LOT MORE, like positioning our elements on the page.

- Classes are just a list of attributes, which is like the color, background-color. Except it may not be text styling but like height, width, column, row.
-

Let’s make a CSS file first,

```
<> example.html
🖼️ myImage.jpg
# styles.css
```

You make a CLASS like this, then you put your attributes in there.

```
.classname {
  color: red;
}
```

```
# styles.css  X
Learning part > # styles.css > ...
1  .classname {
2    color: red;
3  }
4
```

This class is then put into our HTML element, or the entire division surrounding that HTML element.

- First, we need the HTML file to access the CSS information.
- Put what’s below at the top of your HTML file.

```
<link rel="stylesheet" href="styles.css">
```

AS YOU CAN SEE, this achieves the same as in-line. But CLEANER; if we had too many attributes to edit, doing it in-line would be TOO MUCH.

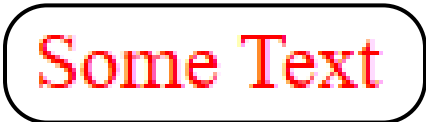
Then let's add this class to our text.

```
<> example.html X
Learning part > <> example.html > ...
1  <link rel="stylesheet" href="styles.css">
2
3  <div>
4  |   <p>Some Text</p>
5  </div>
```

Look at line 3, we're going to add this *IN-LINE* thing called **a class**. Remember how before we had to edit our properties inside, NOW we can do it elsewhere and apply it here.

- `<div class="classname">`

Now our text looks like this.



Some Text

NOW, let's get to attributes that affect positioning; (go back to the CSS file.)

This is why I mentioned DIVISIONS, because we want to have our elements like our *images, text, buttons, links* or whatever **to be organized into certain positions**.

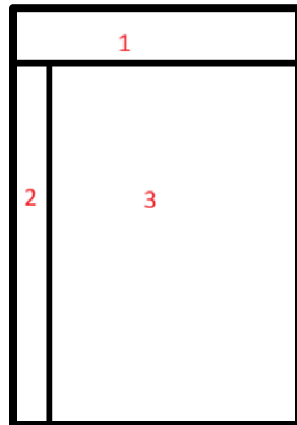
- When we position stuff on a page, we use something called a **BOX MODEL**, which literally just means splitting up a page into boxes.
 - Think of QUADRANTS on a math grid, *but there can be more than 4; and they DON'T have to be the same size*.

The way we achieve splitting our page into boxes, or quadrants is via a **GRID**

- A grid is something called a DISPLAY ATTRIBUTE you add to a class; in that class; you specify the row & column dimensions you want.
 - Then your divisions (the HTML div's) fill those quadrants in order of as they show up in your HTML code. (i.e. row 1 column 1, then row 1 column 2)

Let me show you an actual example.

- Im going to split my entire page into 3 div's one to hold my **TITLE**, one to hold my **TABLE OF CONTENTS** and one to hold the actual **CONTENT**. JUST like this image.
 - This is called the BOX MODEL; separating a bigger space (not just the entire page, maybe another division) into boxes.
 - (Please empty out both the HTML and CSS file, aside from the <link> in the HTML file.)



First, we make our **SKELETON division** that all of our **divisions** fits under,

- Go to the CSS file; make a class called skeleton.

```
.skeleton {}
```

Here, we make our grid; with the *columns and rows*.

```
.skeleton {  
  display: grid;  
  grid-template-columns: 250px 1fr;  
  grid-template-rows: 100px 1fr;  
}
```

The way **this works**, is we make the skeleton a grid. So we can create columns & rows.

- The columns take inputs for the *WIDTH* of the column, and the amount of columns is the same as how many **arguments you gave**. For rows it's the same, but the arguments are *HEIGHT*.
 - **You also see the 2nd** argument is "1fr"; it just means "FREE ROOM" so it'll just fill up the room that the 1st argument didn't.

So, now we have **2 rows**, and **2 columns**. So our we put our **OTHER DIVISIONS** inside this skeleton division.

So to can SEE our divisions; let's name our classes, remember I wanted a title, a table of contents and a content section.

```
.title {  
  background-color: #ccc;  
}  
.table-of-contents {  
  background-color: #6f6363;  
}  
.content {  
  background-color: #613737;  
}
```

Now let's go back to HTML, make our skeleton *DIVISION*, the title *DIVISION*, the table of contents *DIVISION*, and the content *DIVISION*.

```
<link rel="stylesheet" href="styles.css">
```

- This is the CSS we took.

```
<div class="skeleton">
```

- This is the **SKELETON** *division*; that **established** the rows & columns.

```
<div class="title"> Random Text
```

- This is the **TITLE** *division*; fills row 1 column 1.

```
</div>
```

```
<div class="table-of-contents"> Random Text
```

- This is the **TABLE OF CONTENTS** *division*; fills row 1 column 2.

```
</div>
```

```
<div class="content"> Random Text
```

- This is the **CONTENT** *division*; fills row 2 column 1.

```
</div>
```

```
</div>
```

As you can see, the **DIVISIONS**, are just placed in the **NEXT** available quadrant.

- Now, our page looks like this.



But, I have a couple changes I want. I want the TITLE section to **take up the ENTIRE first row**. And I want my Table of Contents, as well as my Content to stretch fully.

- We achieve that **FIRST criteria** via an attribute called `grid-column`

So we head over to our class `"title"`, and we add the attribute `grid-column`

- Which TAKES in 2 arguments; Which column to START at, and which to end at.

```
.title {  
  background-color: #ccc;  
  grid-column: 1 / -1;  
}
```

- Start at column 1, end at column -1; -1 means LAST ONE, if you've done indexing in lists; IT'S the same.

The SECOND criteria is to make the HEIGHT fill up the entire screen. And I wanted to do that for the 2nd & 3rd divisions.

- That is done via a property called **VIEW PORT HEIGHT**; which just means the entire rest of the screen.

```
.table-of-contents {  
  background-color: #6f6363;  
  height: 100vh; /*Take up 100% of the rest of the screen*/  
}  
  
.content {  
  background-color: #613737;  
  height: 100vh;  
}
```

Now this is our page. In the next part we'll actually put stuff in these divisions.

